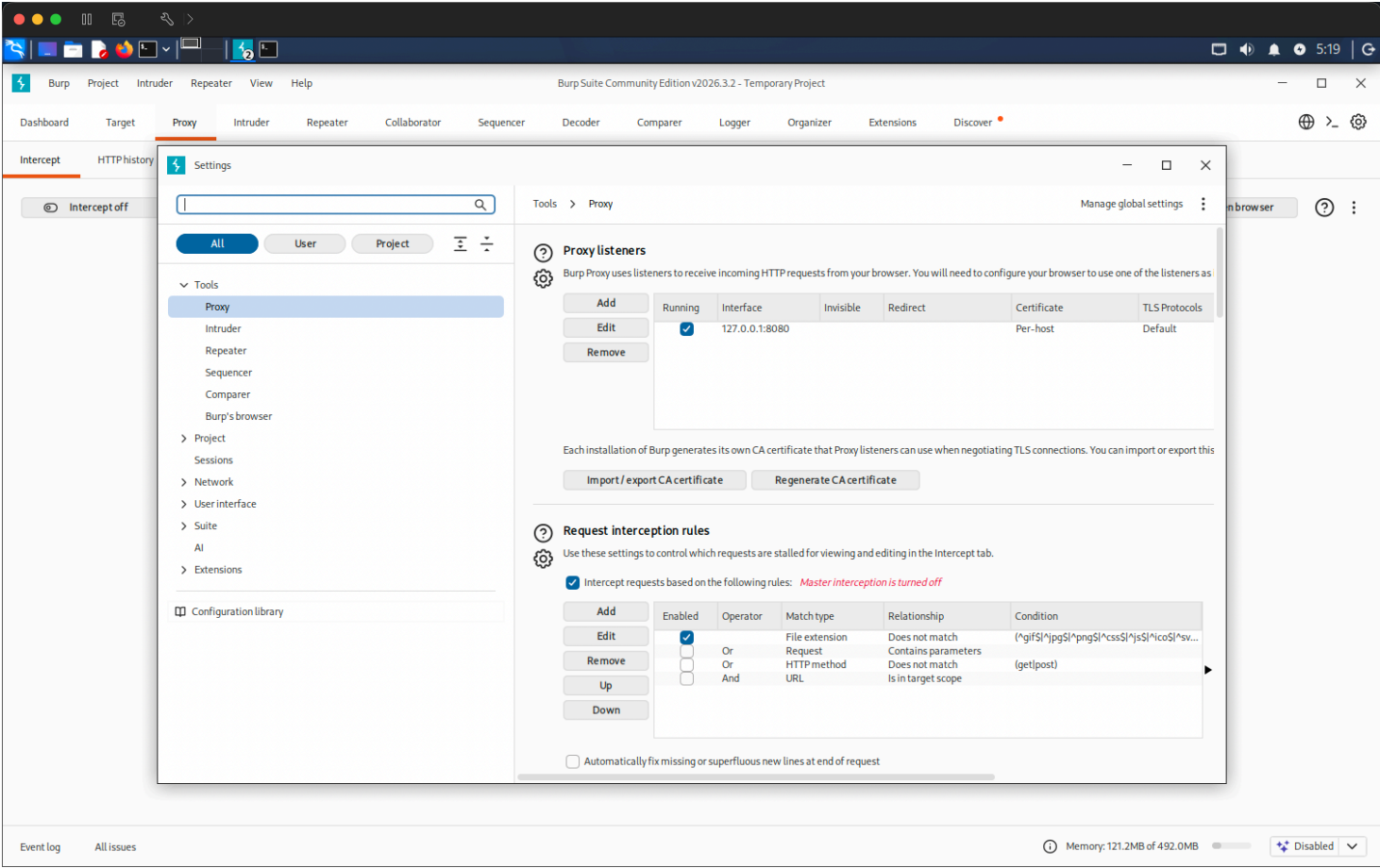


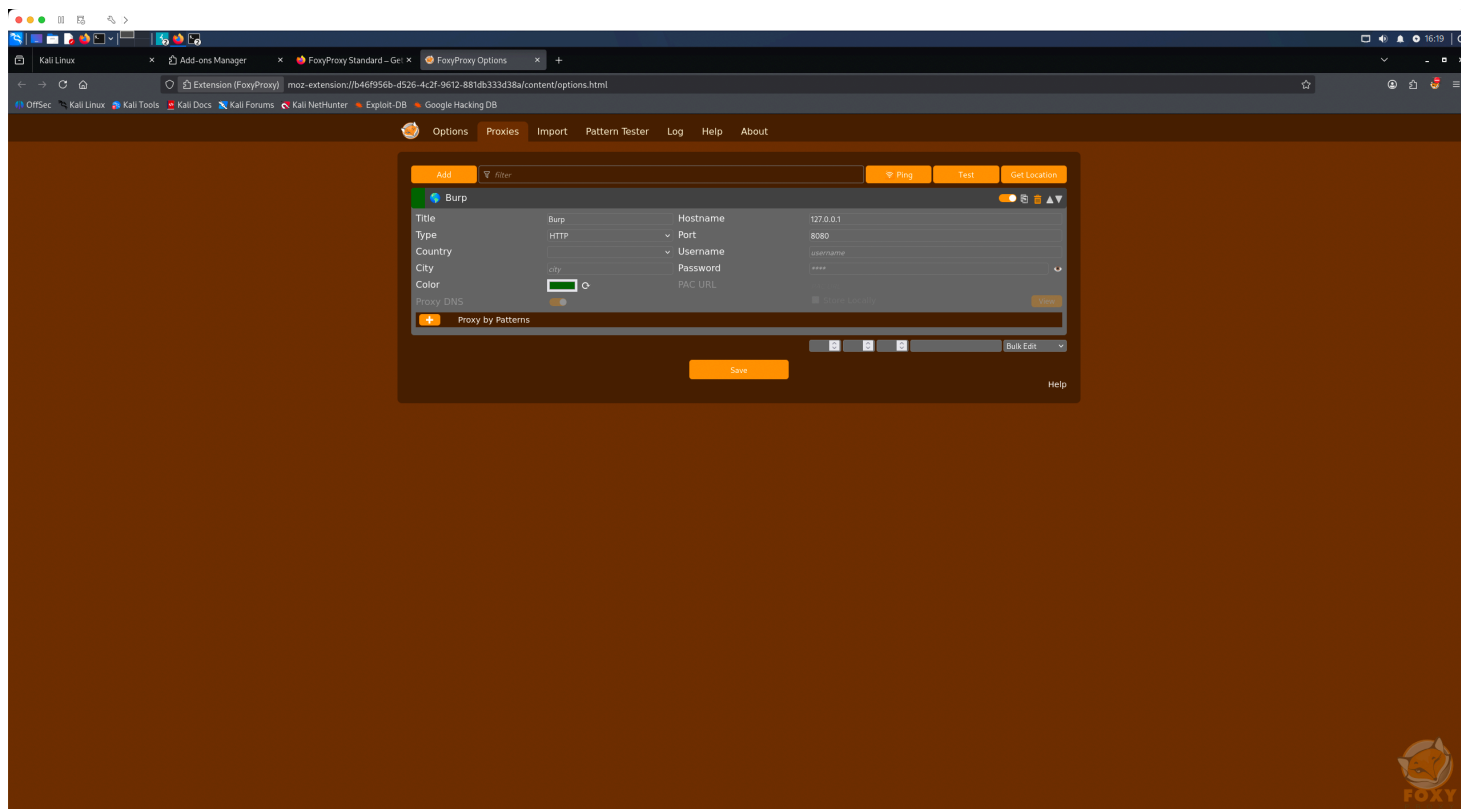
Web Hacking Assignment Report

Author: Zhenyu Song (zhenyus4@uci.edu)
Date: 06/03/2026
Environment: Kali Linux VM
Target application: OWASP Juice Shop at <http://127.0.0.1:42000>

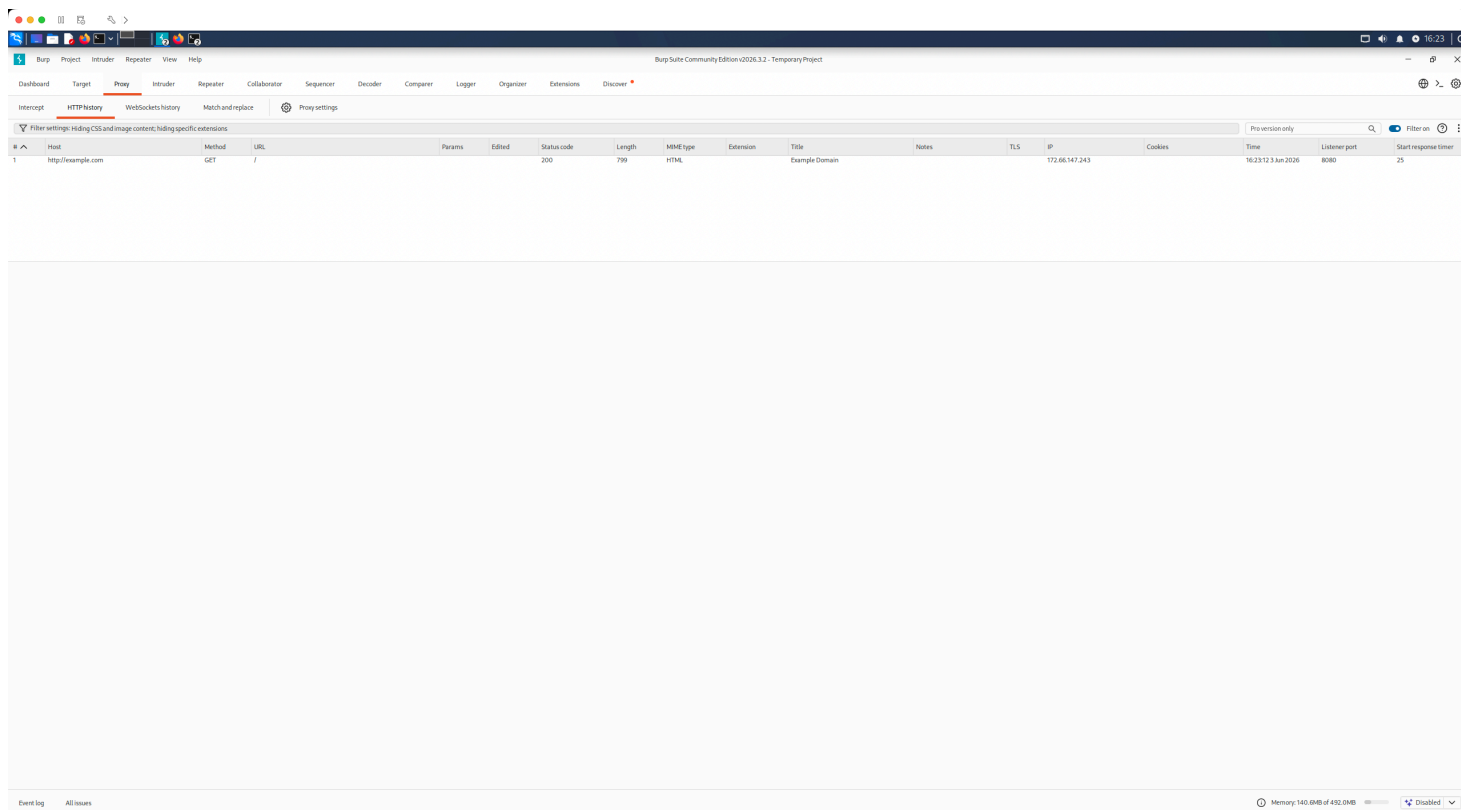
1. Web Application Proxy

For this assignment I used **Burp Suite Community Edition** as the web application proxy and **FoxyProxy** as the browser proxy manager. Burp was configured to listen on `127.0.0.1:8080`, and FoxyProxy was configured to send browser traffic to that address when enabled.

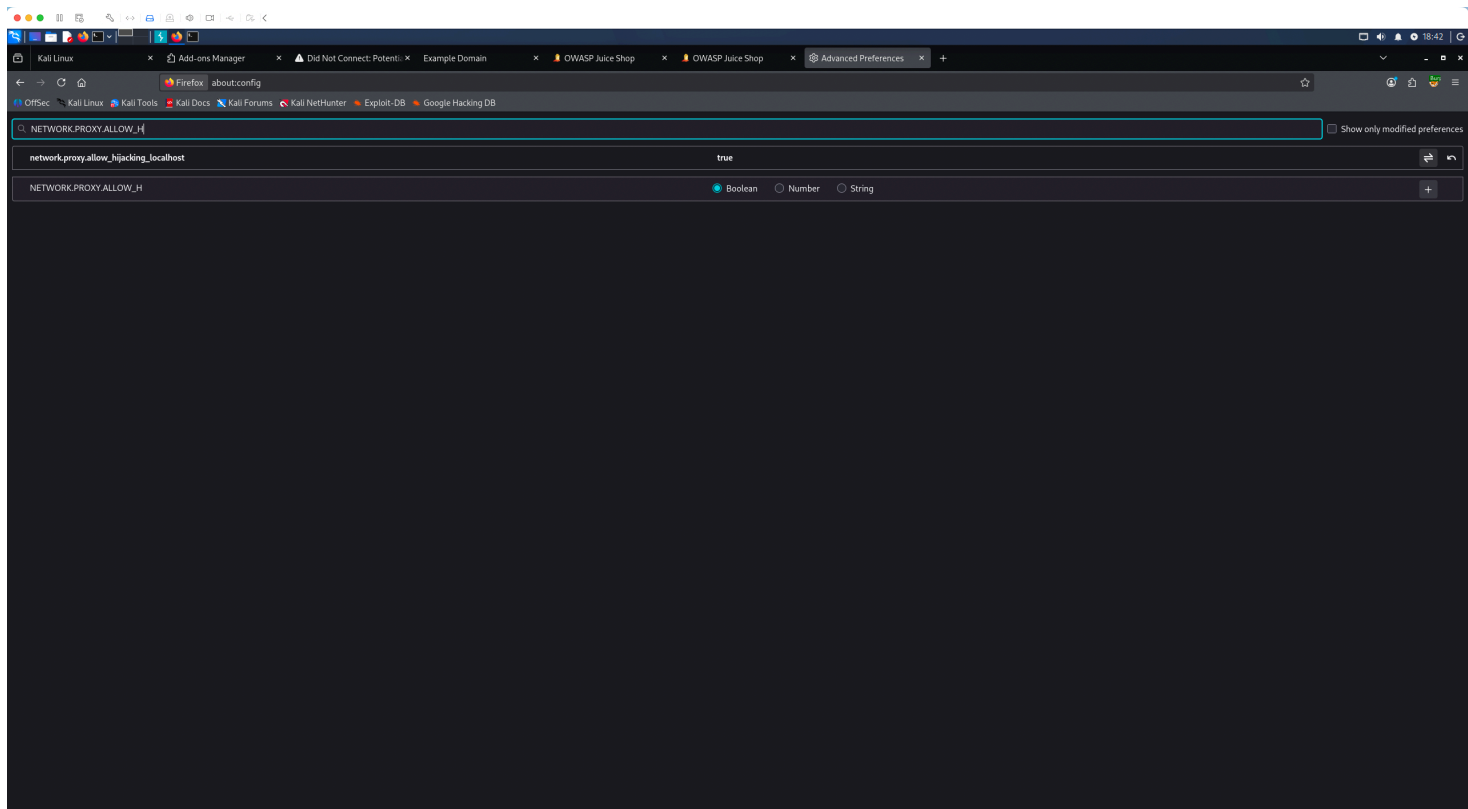




I verified that the proxy was working by enabling FoxyProxy, browsing to the Juice Shop site, and confirming that HTTP requests appeared in Burp Proxy HTTP history. I also tested toggling the proxy off to make sure the browser could return to direct network access.



The main proxy setup challenge was getting Firefox to send local Juice Shop traffic to Burp. FoxyProxy was correctly configured for `127.0.0.1:8080`, and Burp successfully captured external test traffic such as `example.com`, but requests to `http://127.0.0.1:42000` did not initially appear in Burp HTTP history. I first tried Burp's built-in browser, but this Kali VM displayed `Burp browser is not available`. I then checked Firefox's proxy settings and found that the visible `No proxy for` field was empty, but Firefox still had the advanced preference `network.proxy.allow_hijacking_localhost` set to `false`. After changing this preference to `true` in `about:config`, Firefox was allowed to proxy local `127.0.0.1` traffic through Burp.



I chose Burp Suite because it is widely used for web application testing, easy to configure for manual request inspection, and already familiar in many security labs.

Pros: Burp makes it easy to inspect requests, modify parameters, replay traffic, and understand how the browser communicates with the backend. It also works well with FoxyProxy for quickly switching proxy settings.

Cons: The Community Edition has limitations compared with the paid version, especially around automation and scanning. It can also interrupt normal browsing if intercept is left enabled.

2. OWASP Juice Shop Installation

I first attempted to install OWASP Juice Shop from the Kali package repository:

```
sudo apt update
sudo apt install juice-shop
```

This failed with `Unable to locate package juice-shop`. The Kali VM was using the ARM (`arm64`) package repository, while the Kali Juice Shop package was not available for this architecture in my repository configuration. I documented this as the main installation challenge.

```
Session Actions Edit View Help
kali@kali:~$ ss -nltn | grep 8080
tcp        LISTEN     0      50      :::ffff:127.0.0.1:::8080      *:*      users:(("java",pid=22556,fd=33))

kali@kali:~$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main arm64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main arm64 Contents (deb) [49.7 MB]
Fetched 70.6 MB in 13s (5509 kB/s)
1009 packages can be upgraded. Run 'apt list --upgradable' to see them.

kali@kali:~$ sudo apt install juice-shop
E: Unable to locate package juice-shop

kali@kali:~$
```

To continue the lab without changing the VM, I installed and ran Juice Shop with **Podman** using the official container image:

```
sudo apt install podman
podman run -d --name juice-shop -p 127.0.0.1:42000:3000 docker.io/bkimminich/juice-shop
```

I verified that the container was running with:

```
podman ps
```

I also verified that Juice Shop was listening locally on port 42000 with:

```
ss -ltnp | grep 42000
```

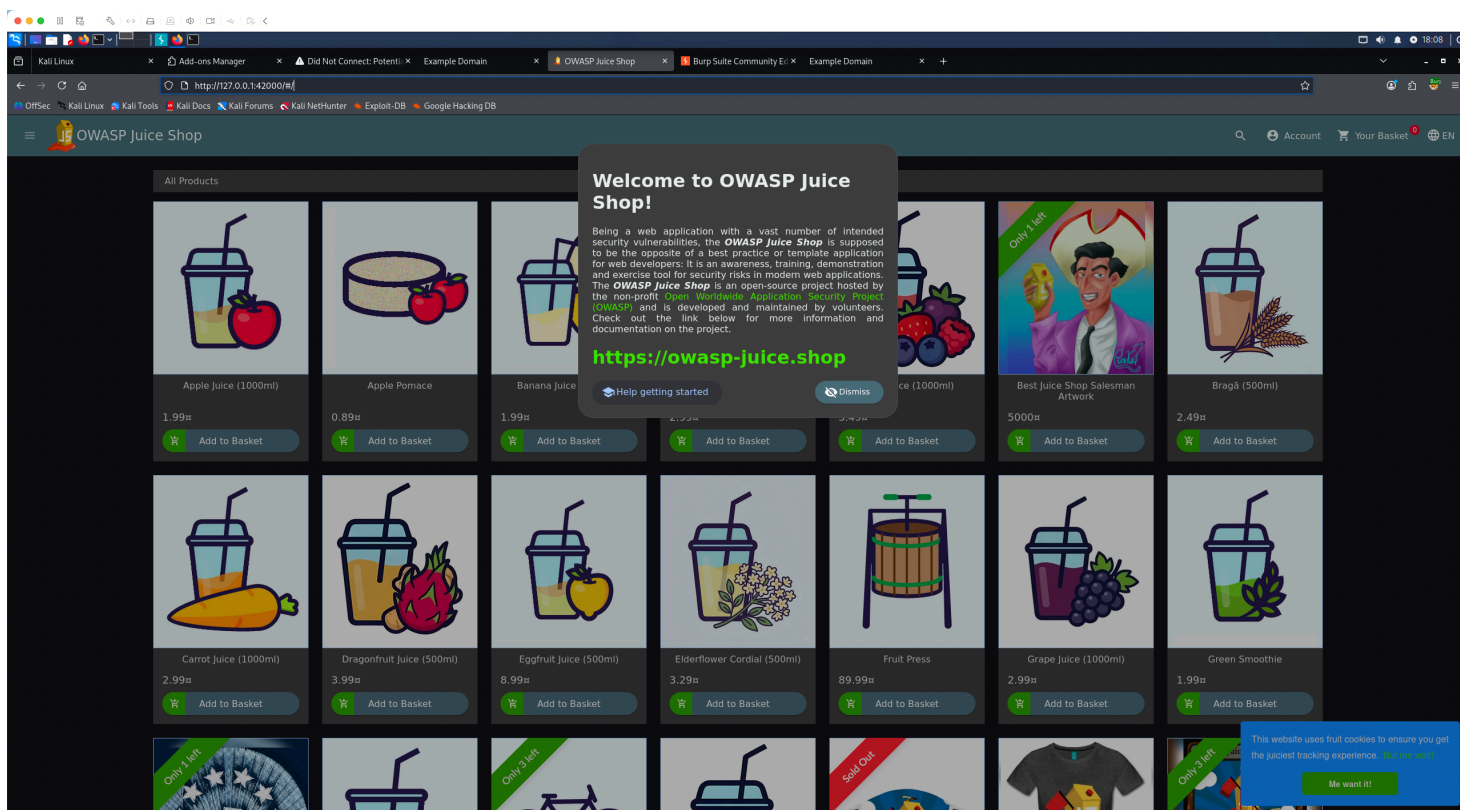


```
Session Actions Edit View Help
Selecting previously unselected package podman.
Preparing to unpack ./lib-podman-3.8.2+ds1-2.arm64.deb.
Unpacking podman (3.8.2+ds1-2)...
Selecting previously unselected package python3-protobuf.
Preparing to unpack ./python3-protobuf_3.21.12-15+b1_arm64.deb.
Unpacking python3-protobuf (3.21.12-15+b1)...
Selecting previously unselected package python3-pycrui.
Preparing to unpack ./python3-pycrui_4.2.4_all.deb.
Unpacking python3-pycrui (4.2.4)...
Selecting previously unselected package slurpnetns.
Preparing to unpack ./slurpnetns_1.3.3-1+b1_arm64.deb.
Unpacking slurpnetns (1.3.3-1+b1)...
Setting up libblake3-0 (1.6.2-1)...
Setting up libblake3-arm64 (1.6.2-1-2)...
Setting up libcompell-arm64 (4.2.4)...
Setting up golang-github-containers-image (0.39.2-1)...
Setting up common (2.1.13+ds1-2+b1)...
Setting up catusql (0.2.1-3+b388)...
Setting up netavark (1.16.1-1)...
netavark-dhcp-proxy.service is a disabled or a static unit, not starting it.
netavark-dhcp-proxy.socket is a disabled or a static unit, not starting it.
netavark-firewall-reload.service is a disabled or a static unit, not starting it.
Setting up netavark-dns (2.16.0-1)...
Setting up libblake3-arm64 (4.9.3-1)...
Setting up passt (0.6.0+git20200226.088c51e-1)...
Setting up crui (4.2-4)...
Setting up fuse-overlays (1.14-1+b2)...
Setting up golang-github-containers-common (0.67.1-2)...
Setting up python3-protobuf (3.21.12-15+b1)...
Setting up python3-pycrui (4.2-4)...
Setting up slurpnetns (1.3.3-1+b1)...
Setting up libcrui-arm64 (4.2-4)...
Setting up crui (1.27.1-1)...
Setting up widmap (1.14.19.3-2)...
Setting up containers-storage (1.62.0-1)...
Setting up podman (3.8.2+ds1-2)...
Setting up buildah (1.40.2+ds1-1)...
Processing triggers for libc-bin (2.42-13)...
Processing triggers for man-db (2.11.1-1+b1)...
Processing triggers for kali-menu (2025.1.5)...

(kali@kali) ~$ podman run -d --name juice-shop -p 127.0.0.1:42000:3000 docker.io/bkimminich/juice-shop
Trying to pull docker.io/bkimminich/juice-shop:latest ...
Getting image source signatures
Copying blob 278928e0c50f done |
Copying blob 4e796f6e1b95 done |
Copying blob b46ef1bfc6ba done |
Copying blob c172f21a10df done |
Copying blob ba242723c53f done |
Copying blob d04b1b0ecac done |
Copying blob 7c12895b777b done |
Copying blob 3214acfa1c0b done |
Copying blob 5263bf75a18 done |
Copying blob d04ab72d177 done |
Copying blob 3839eae01f6 done |
Copying blob e8bdc55fadc done |
Copying blob b6f6f1a3a6 done |
Copying blob f3e19d19e1f done |
Copying blob 2316fdcc6ba done |
Copying blob 51b2cf5d8a5 done |
Copying blob 887205a8f795 done |
Copying blob 7245b19ca109 done |
Copying blob 35d8e5f29ac4f done |
Copying blob f44321d1d1d done |
Copying blob 278ac0e295e done |
Copying blob 1597f10fca2 done |
Copying blob c7d995a52779 done |
Copying blob bc251321a7d done |
Copying config 4f4071a3cf done |
Writing manifest to image destination
b1eb4531a3eef4b026fd1f41a0f0a0e7c78aebb0ba93a9b2f01a3e7d9f

(kali@kali) ~$ ss -ltnp | grep 42000
LISTEN 0      128          127.0.0.1:* 42000          0.0.0.0:*    users: ("pasta",pid=5055,fd=152)
```

I accessed the application at `http://127.0.0.1:42000` .



Because I used the container method, Juice Shop was managed from the command line with Podman instead of the Kali helper scripts. I stopped and started it with:

```
podman stop juice-shop
podman start juice-shop
```

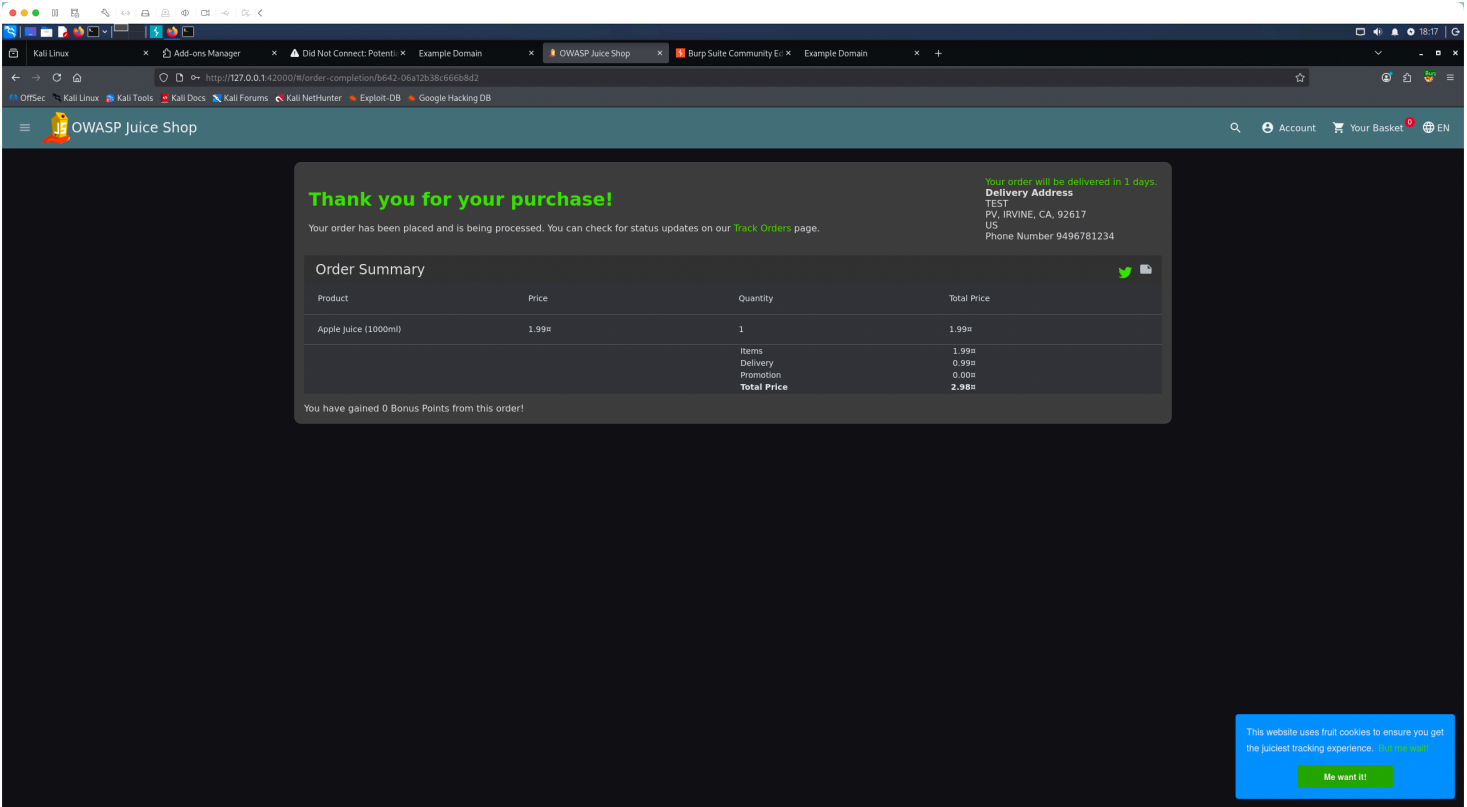
After stopping it, I verified that port `42000` was no longer listening.

Although my installation used Podman, I also reviewed the official Kali helper commands that would normally be installed under `/usr/bin` . The current Kali package documents `juice-shop-start` and `juice-shop-stop` ; the older `juice-shop` command is deprecated. These helper commands manage `juice-shop.service` , whose main process is `npm start` , and the packaged service exposes Juice Shop on TCP port `42000` . My `podman run -d -p 127.0.0.1:42000:3000 setup plus podman stop/start juice-shop` was functionally equivalent for the lab, but used container isolation instead of running Node directly on the Kali host.

From an attack surface perspective, running Juice Shop exposes a deliberately vulnerable web application on the configured local port. Binding it to `127.0.0.1:42000` kept the service limited to the local VM, which reduced unnecessary network exposure while still allowing browser and proxy testing.

3. Juice Shop Tutorials

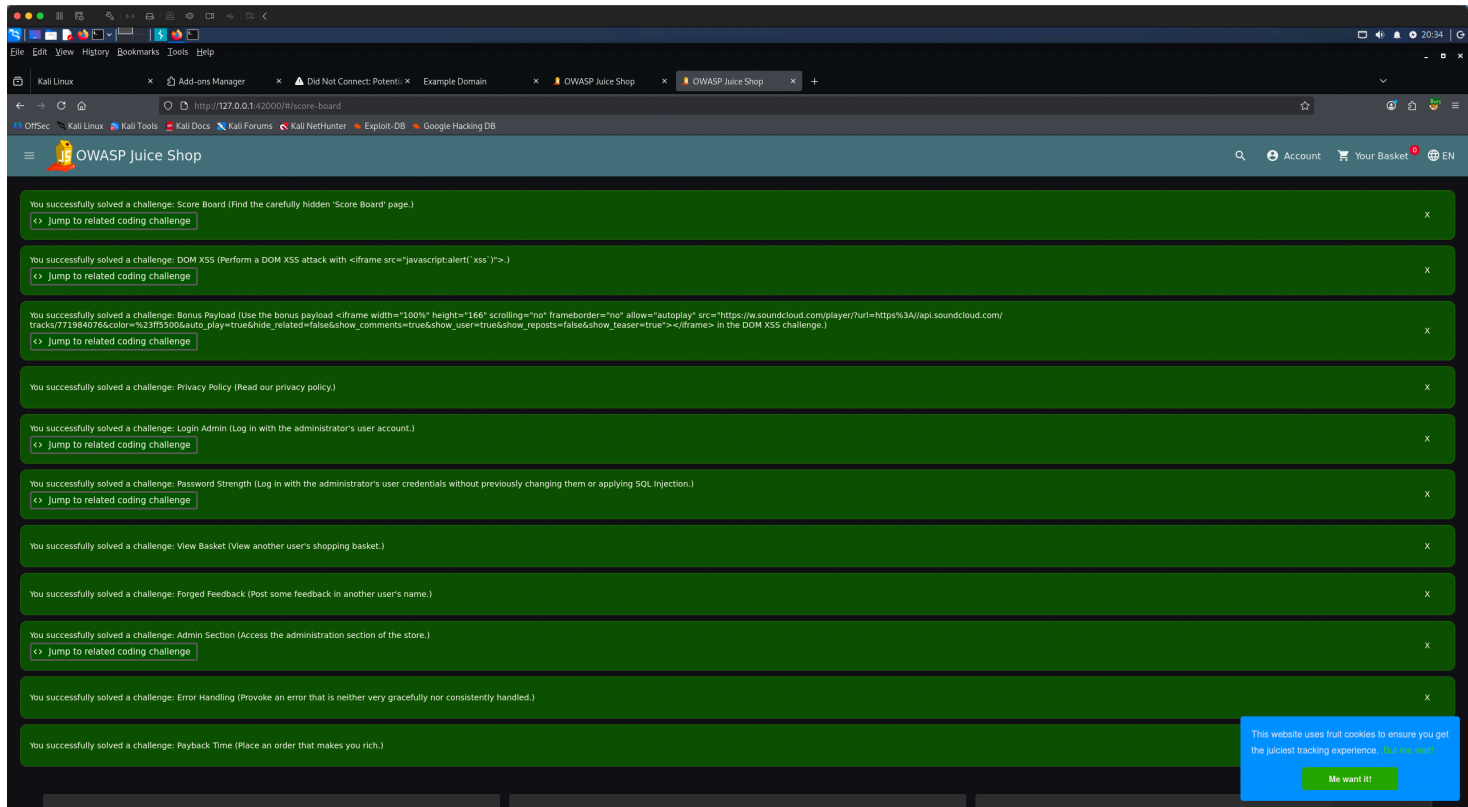
After starting Juice Shop, I created a test user, logged in, browsed products, added items to the basket, completed a purchase with fictitious payment data, reviewed the order details, and explored areas such as feedback, complaint, and support chat.



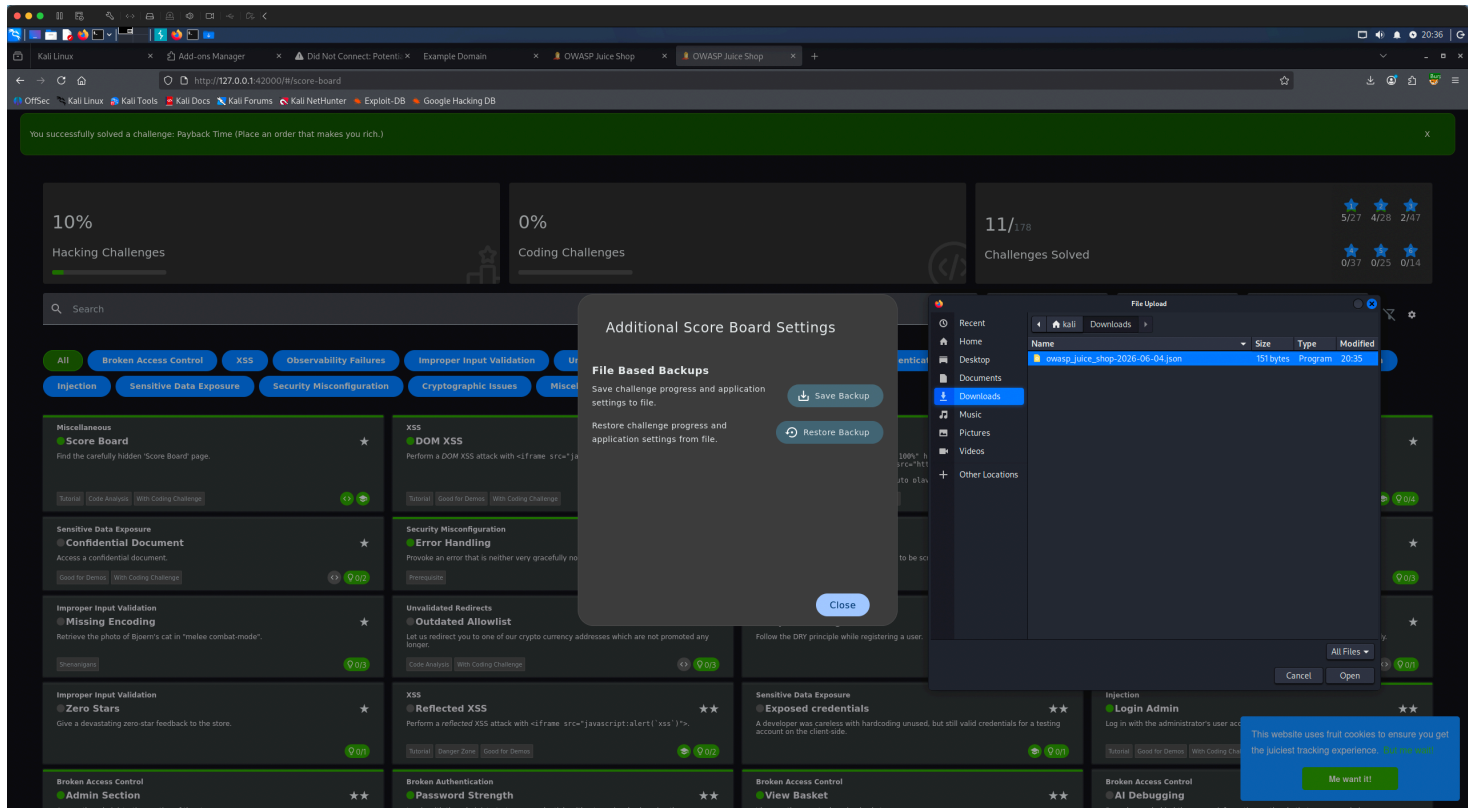
While browsing the site, I inspected traffic with Burp and the browser developer tools. This showed API requests and response data that were not always visible in the page UI, which helped explain how the single-page application communicates with the backend.

I completed the required tutorials from the Score Board:

- Scoreboard
- DOM XSS
- Bonus Payload
- Privacy Policy
- Login Admin
- Password Strength
- View Basket
- Forged Feedback



I also tested the progress backup and restore feature. I saved the current challenge state to a file, then restored from that saved file to confirm that challenge progress could be recovered.



Juice Shop is useful because it combines realistic web application behavior with intentionally vulnerable features and built-in progress tracking. Its Score Board gives clear feedback, and the backup/restore feature makes it easier to retry challenges without losing progress. A limitation is that some challenges are intentionally artificial, so the exact exploit steps may not always match a production application, but the underlying security lessons are still useful.

4. Web Hacking Challenges

4.1 Scoreboard

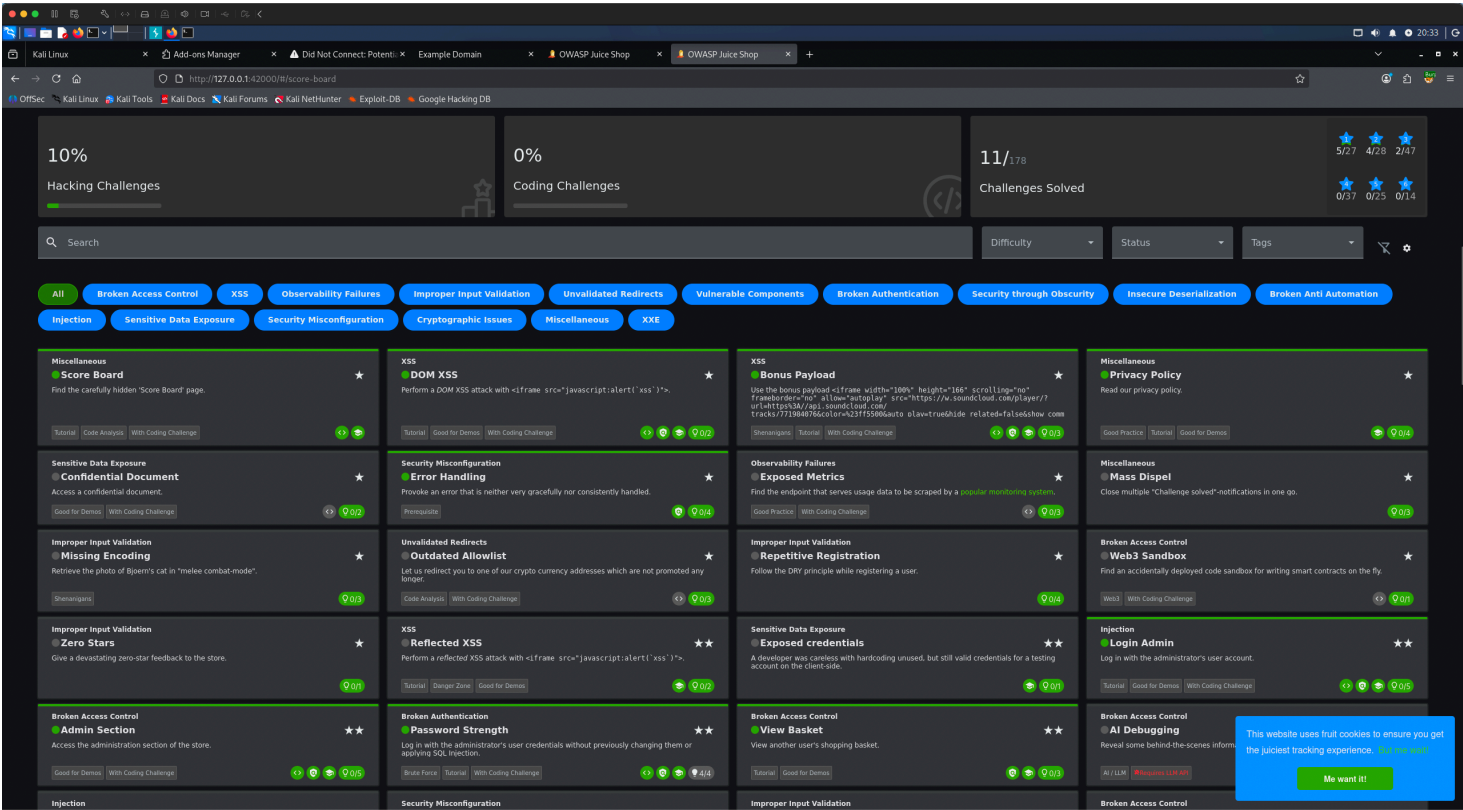
Time spent: 1 minute
Hints or external solutions used: no

My approach was to locate and open the hidden Score Board route, then use it to track solved and unsolved challenges.

What worked: Open `http://127.0.0.1:42000/#/score-board`

What did not work: Browsing only through the normal product UI did not reveal the Score Board link.

Lesson learned: Security testing often starts with discovering application functionality that is not obvious from the main UI.



4.2 DOM XSS

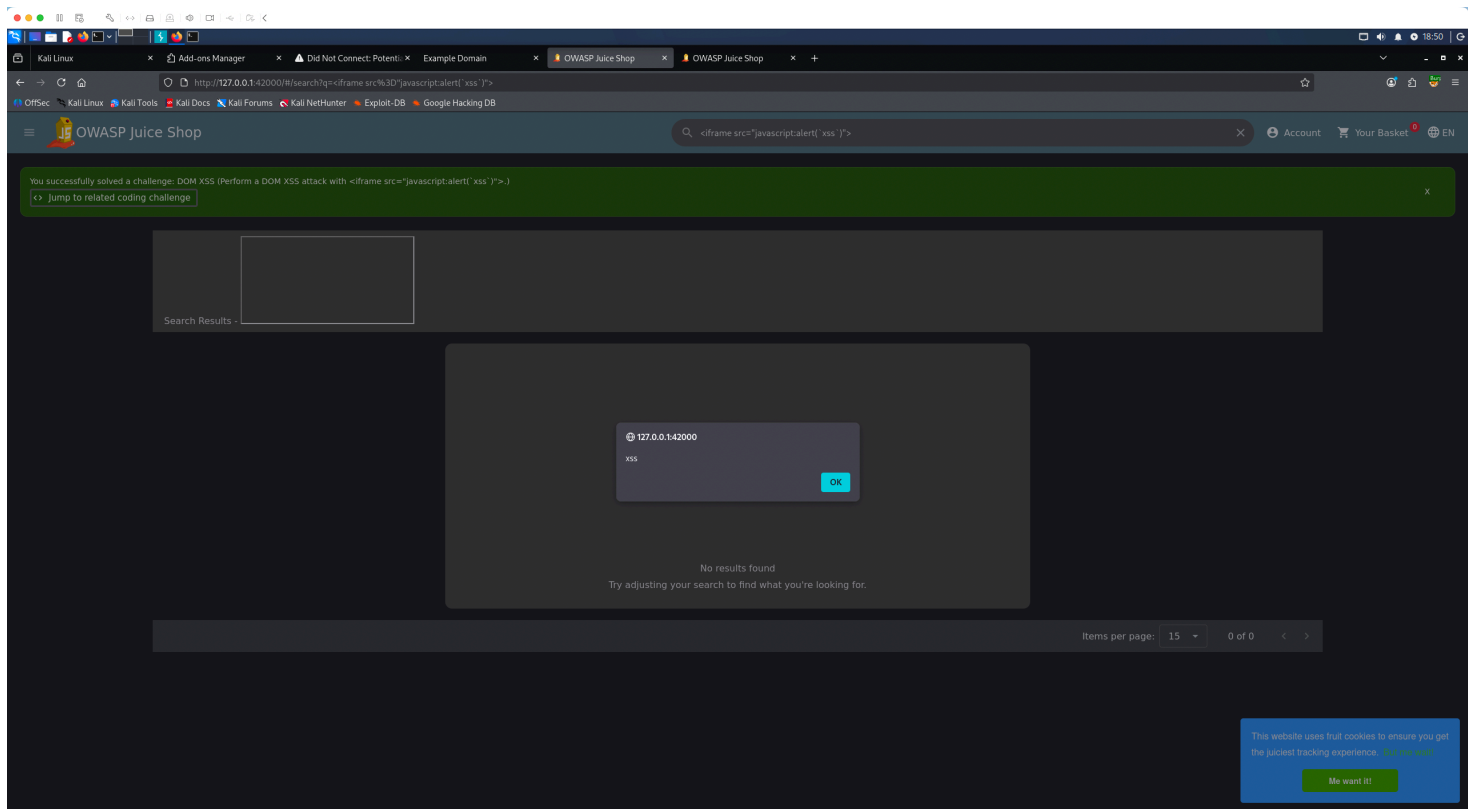
Time spent: 2 minutes
Hints or external solutions used: no

My approach was to identify a user-controlled input that was reflected into the page by client-side JavaScript. I used the search function and tested whether input was rendered as text or interpreted as HTML/JavaScript. The successful solution used a DOM-based script injection payload through the search field.

What worked: Enter `<iframe src="javascript:alert( xss )">` in search box.

What did not work: Normal search strings such as product names were only treated as search terms and did not execute code.

Lesson learned: DOM XSS can happen entirely in browser-side code, so server-side filtering alone is not enough. Output encoding and safe DOM APIs are important defenses.



4.3 Bonus Payload

Time spent: 2 minutes

Hints or external solutions used: no

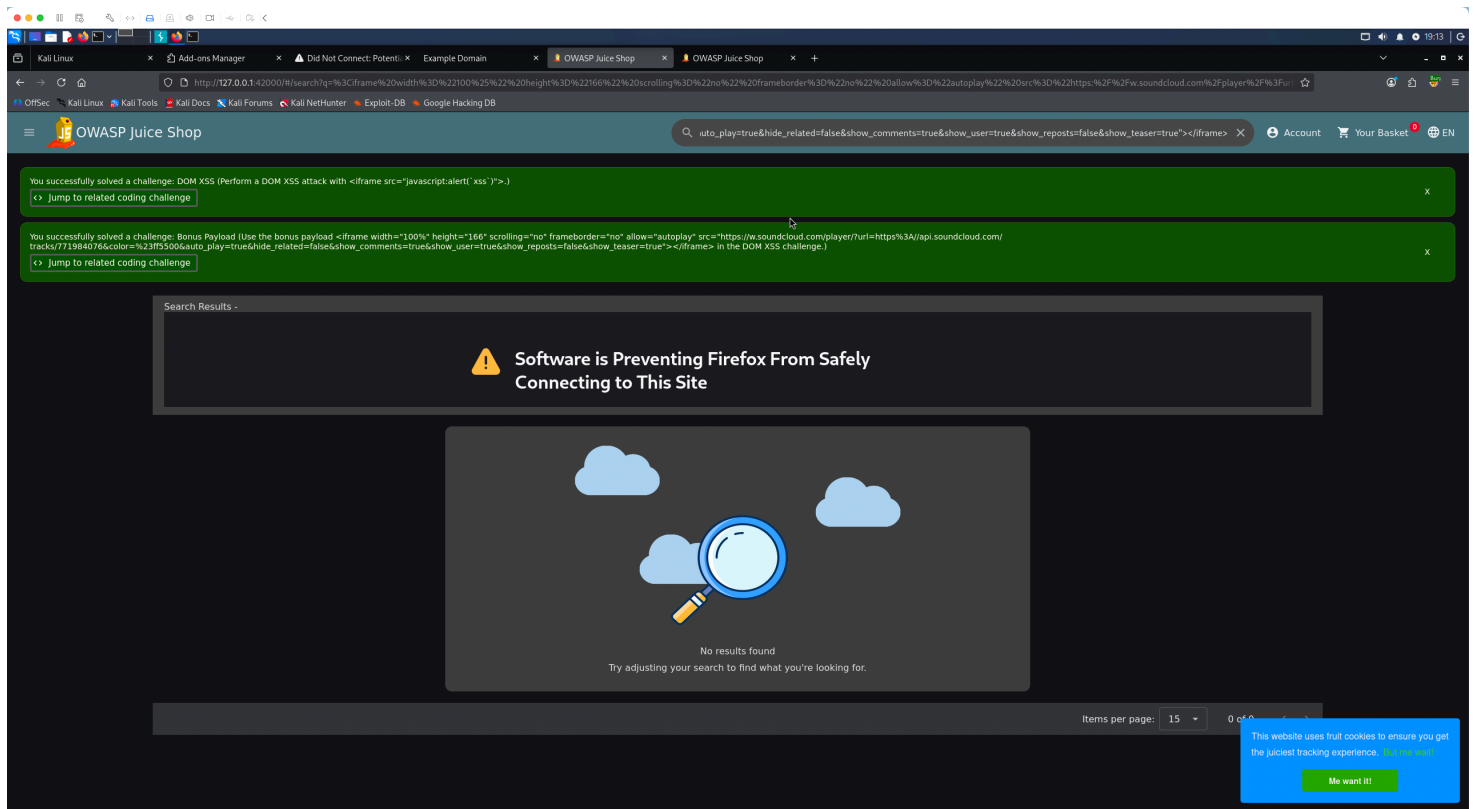
My approach was to follow the tutorial instructions and test a payload variant that Juice Shop recognized as a successful XSS-related solution.

What worked: Enter

`<iframe width="100%" height="166" scrolling="no" frameborder="no" allow="autoplay" src="https://w.soundcloud.com/player/?url=https%3A/.`
in search box.

What did not work: The shorter DOM XSS payload solved DOM XSS but did not satisfy the bonus payload challenge.

Lesson learned: Small payload changes can affect whether injected content executes or is blocked.



4.4 Privacy Policy

Time spent: 1 minute

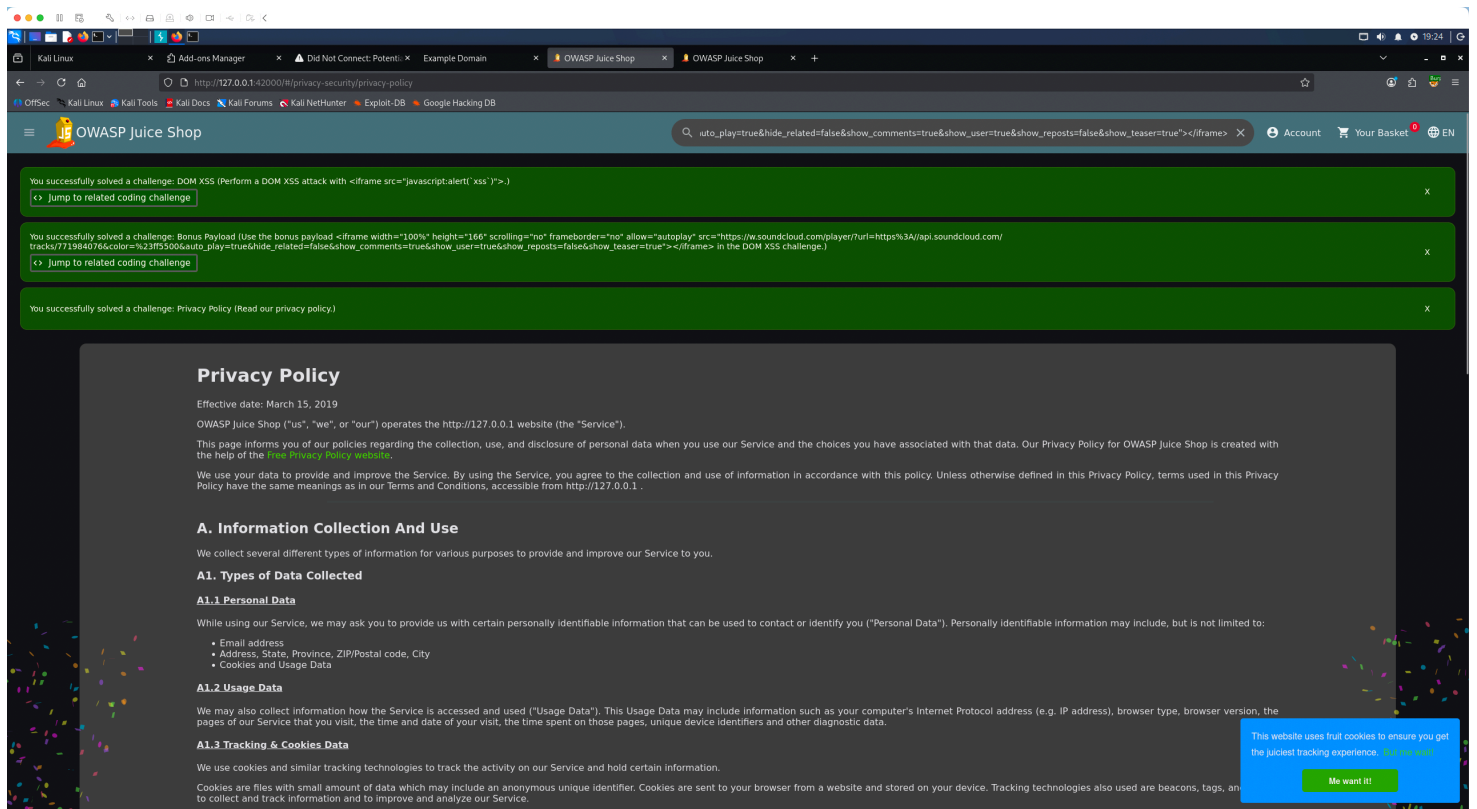
Hints or external solutions used: no

My approach was to explore the application UI and routes until I located the privacy policy page required by the challenge.

What worked: Open Account -> Privacy & Security -> Privacy Policy .

What did not work: Browsing only the product catalog and basket pages did not expose the privacy policy page.

Lesson learned: Informational pages can still reveal useful route and application structure during reconnaissance.



4.5 Login Admin

Time spent: 5 minutes

Hints or external solutions used: no

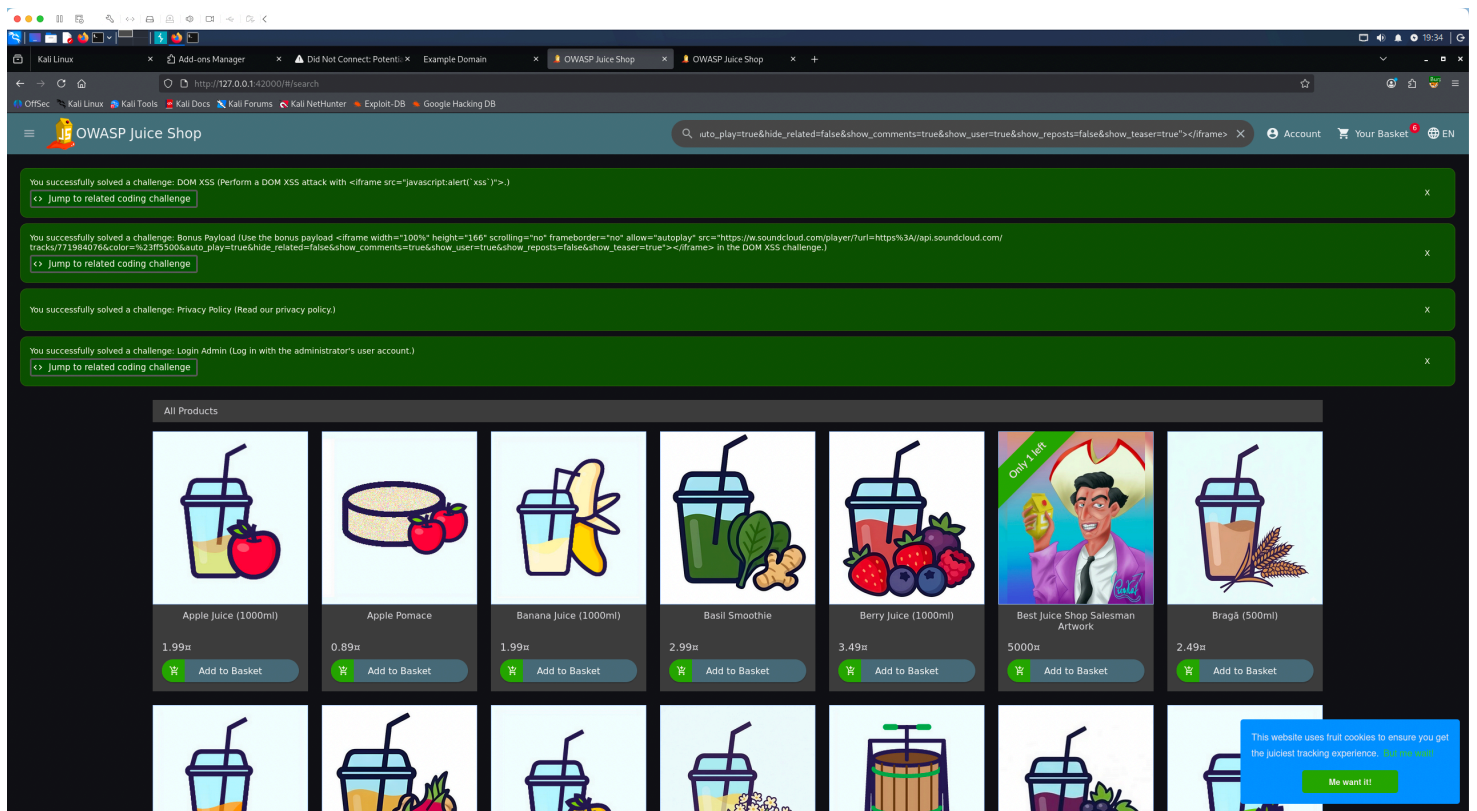
My approach was to test the login workflow and use the guided challenge to understand how weak authentication logic can be bypassed.

```
SELECT * FROM users WHERE email='{user input email}' AND password='{user input password}'
```

What worked: Enter username: ' or 1=1—, password: test in Login page. A SQL injection payload in the email field bypassed login.

What did not work: Normal invalid credentials were rejected.

Lesson learned: Authentication must be enforced server-side with safe input handling and parameterized queries.



4.6 Password Strength

Time spent: 5 minutes

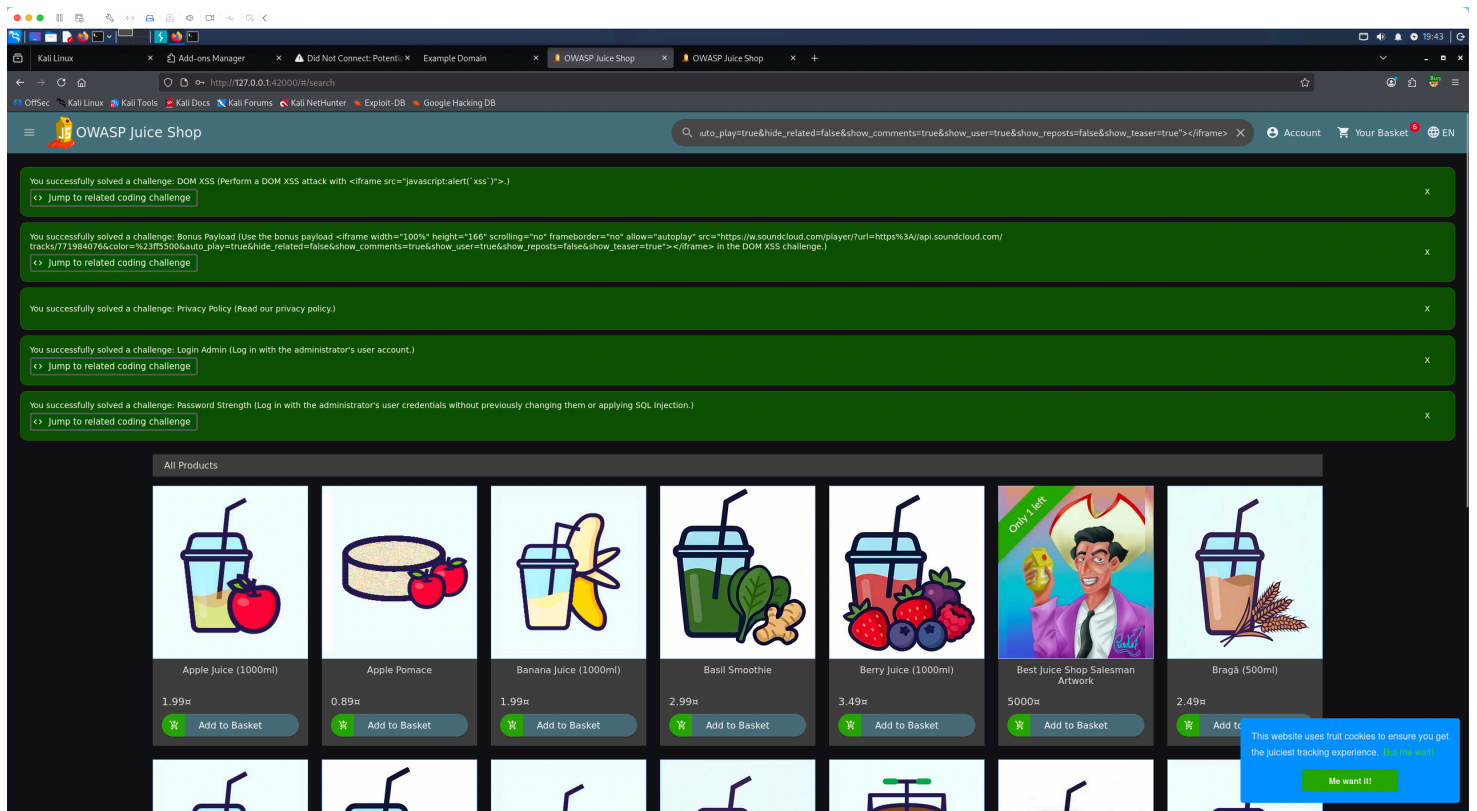
Hints or external solutions used: [yes to get username and password](#)

My approach was to test weak password behavior as directed by the tutorial and confirm that Juice Shop accepted the challenge condition.

What worked: Enter username: `admin@juice-sh.op` , password `admin123` in Login page. The known weak admin password allowed login.

What did not work: Random guessed passwords did not work.

Lesson learned: Password strength requirements reduce account takeover risk but need to be enforced consistently.



4.7 View Basket

Time spent: 10 minutes

Hints or external solutions used: no

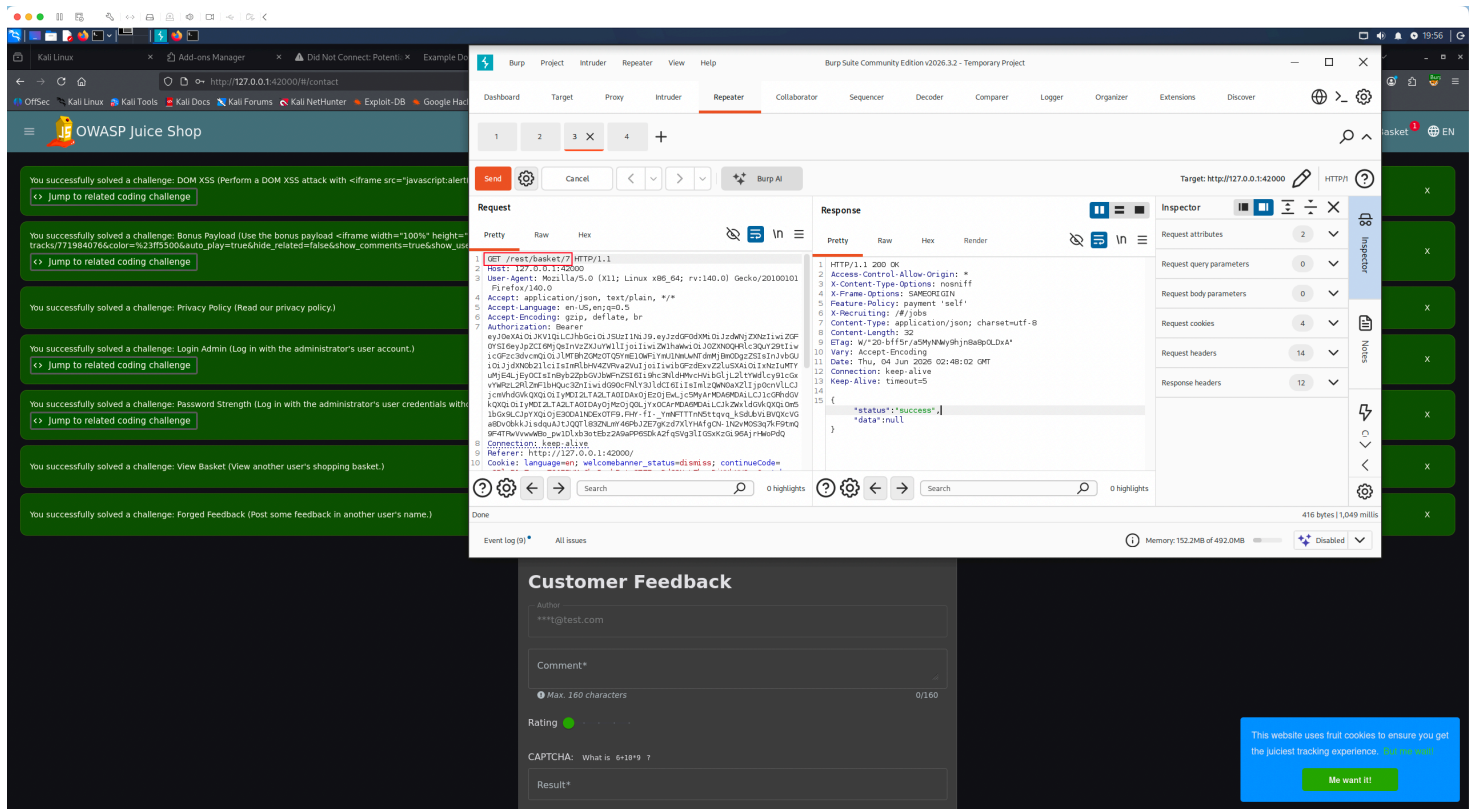
My approach was to inspect basket functionality and understand how basket data is represented in the UI and backend requests.

What worked: Send to server with different basket ID using Burp. Change GET /rest/basket/6 to GET /rest/basket/7 , and then send to repeater.

Changing the basket id in the basket API request exposed another basket.

What did not work: The normal UI only showed my own basket.

Lesson learned: User-specific objects need authorization checks, not just hidden links or client-side restrictions.



4.8 Forged Feedback

Time spent: 5 minutes

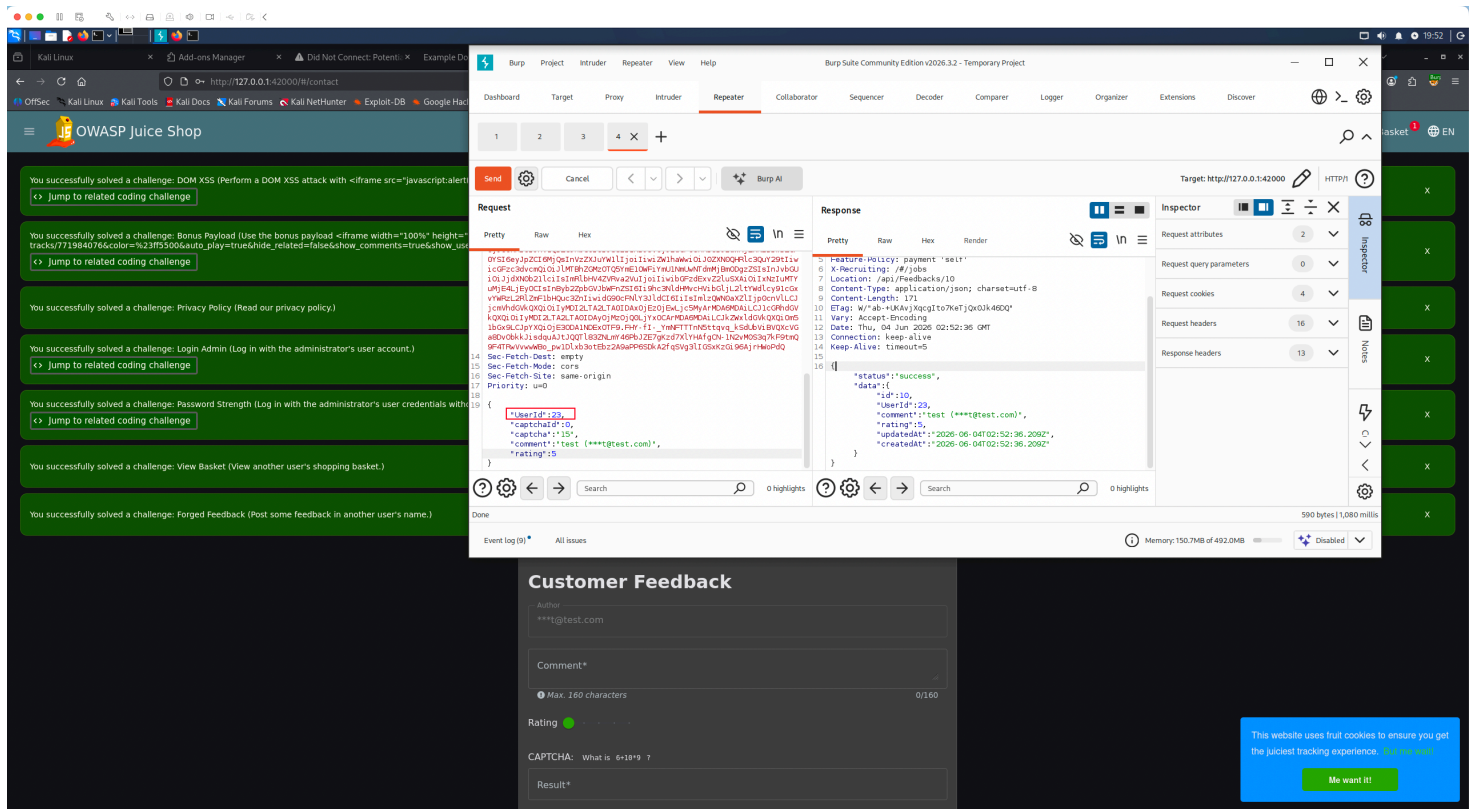
Hints or external solutions used: no

My approach was to submit feedback through the guided workflow and observe how user-controlled metadata could affect the submitted feedback record.

What worked: Send to server with different UserId using Burp. Change the body of POST /api/Feedbacks from "UserId":24 to "UserId":23 , and then send to repeater. Modifying the feedback UserId submitted feedback as another user.

What did not work: Submitting feedback normally through the UI only used my own user identity.

Lesson learned: User-supplied metadata should be validated and tied to the authenticated session instead of trusted directly.



4.9 Admin Section

Time spent: 2 minutes

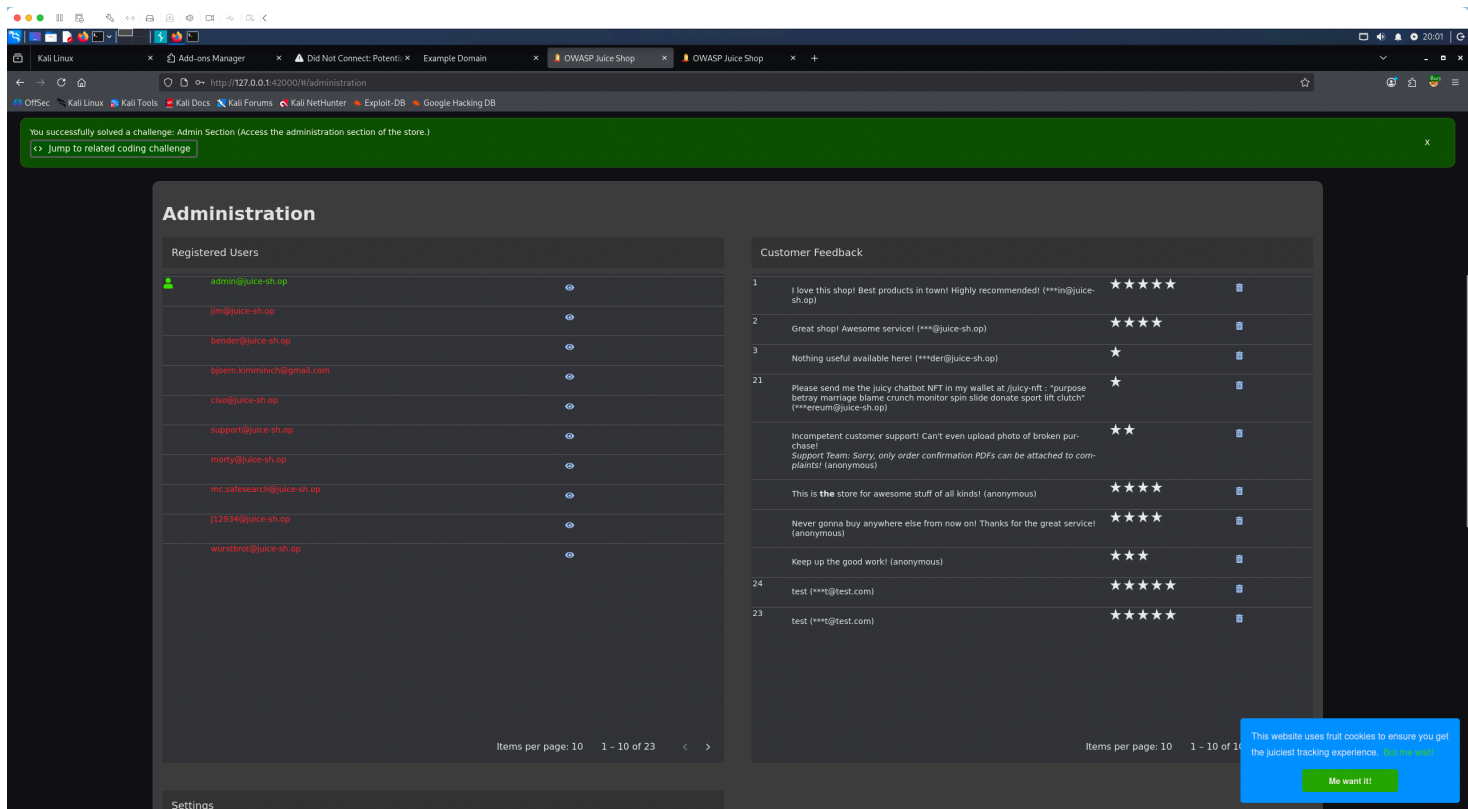
Hints or external solutions used: no

My approach was to explore application routes and client-side behavior to find hidden functionality. After logging in, I inspected available routes and navigation behavior and accessed the administrative section directly once I identified the route.

What worked: Open `http://127.0.0.1:42000/#/administration` with an admin session (username: `'` or `1=1--` , password `test`) solved the challenge.

What did not work: Trying to rely on normal user navigation did not reveal the administration section.

Lesson learned: Hiding admin functionality in the frontend is not an access control mechanism. Sensitive routes must be protected by server-side authorization checks.



4.10 Payback Time

Time spent: 30 minutes

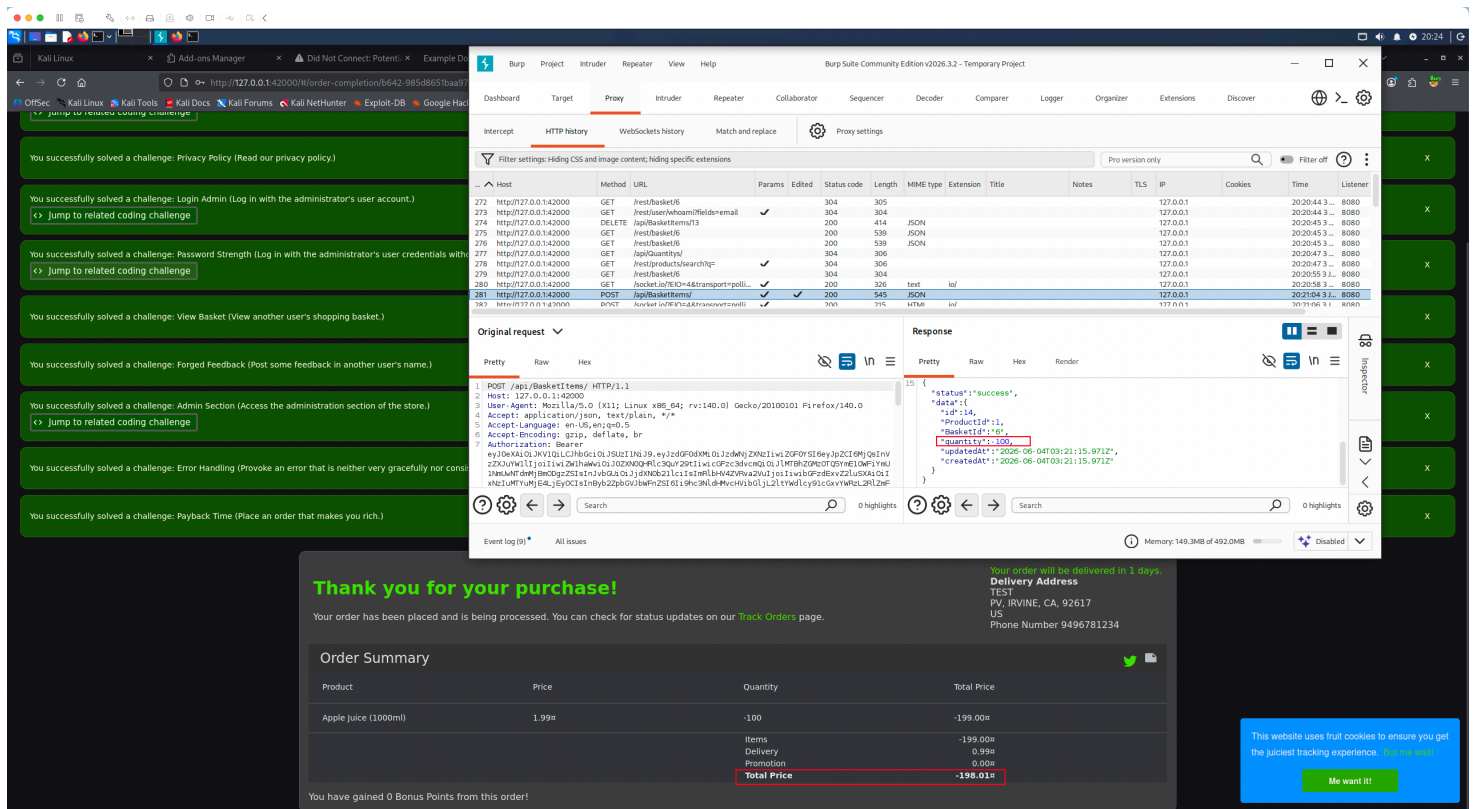
Hints or external solutions used: no

My approach was to use the proxy to inspect and modify the requests involved in basket and order handling. I looked for numeric fields related to quantity, price, or payment values and tested whether the backend accepted manipulated values.

What worked: Turn on Intercept. Modifying basket item quantity to a negative value in `POST /api/BasketItems` changed order behavior.

What did not work: The normal UI did not allow entering a negative quantity directly.

Lesson learned: Business logic values must be validated on the server. A client should not be trusted to provide final prices, balances, or payment amounts.



5. Lessons Learned

This assignment showed how web proxies, browser developer tools, and vulnerable training applications can be used to understand common web security weaknesses. The most important lesson was that many vulnerabilities come from trusting client-side behavior too much. Hidden routes, browser-side input handling, and editable request fields can all become security issues if the server does not validate authorization, input, and business rules.

The assignment also reinforced the value of documenting the testing process. Recording what worked, what failed, and which hints were used makes the result more useful than simply listing final answers.

6. References

- OWASP Juice Shop project: <https://owasp.org/www-project-juice-shop/>
- Kali Juice Shop package documentation: <https://www.kali.org/tools/juice-shop/>
- OWASP Juice Shop companion guide: <https://pwning.owasp-juice.shop/>
- Burp Suite Community Edition: <https://portswigger.net/burp/communitydownload>
- FoxyProxy: <https://getfoxyproxy.org/>
- Password Strength hint source: <https://curiositykillscolby.com/2020/11/15/pwning-owasp-juice-shop-pt-19-password-strength/>